Tema 5 Aprendizaje Automático

Autor: Ismael Sagredo Olivenza

Tema 5.2 Redes de Neuronas

En la semana pasada

- Introducción al Machine Learning
- Tipos de algoritmos de machine learning
- Tipos de problemas de machine learning
- Técnicas de Aprendizaje Automático clásico

Esta semana

- Redes de neuronas artificiales
- El perceptrón multicapa
- Redes recurrentes y parcialmente recurrentes
- Aprendizaje profundo

5.2.1 Introducción a las redes de Neuronas

Al intentar construir máquinas inteligentes surge de forma inmediata como modelo el cerebro humano. Esta tecnología se engloba dentro de la **IA Subsimbolica** que diseña sistemas genéricos, en este caso, intentando modelizar el funcionamiento dle propio cerebro humano.

Las redes de neuronas se basan en los trabajos realizados por **Warren McCulloch** y **Walter Pittis** en 1943.

Apoyándose en la teoría de la computación de Allan Touring y los estudios sobre la neurología, diseñaron un sistema basado en neuronas artificiales que simulaba el funcionamiento de las neuronas biológicas

Demostraron que cualquier función de cómputo podía calcularse sobre una red de dichas neuronas y que éstas, incluso, podrían llegar a aprender.

Frank Rosenblatt (1957) generalizó el modelo de neuronas artificiales de McCulloch-Pittis añadiéndole aprendizaje y creando el Perceptron.

En 1986 se desarrollo del algoritmo de **Retropropagación** (Rumelhart, Hinton y Willians en 1986) y a partir de entonces las Redes de Neuronas se han ido aplicando a diferentes problemas

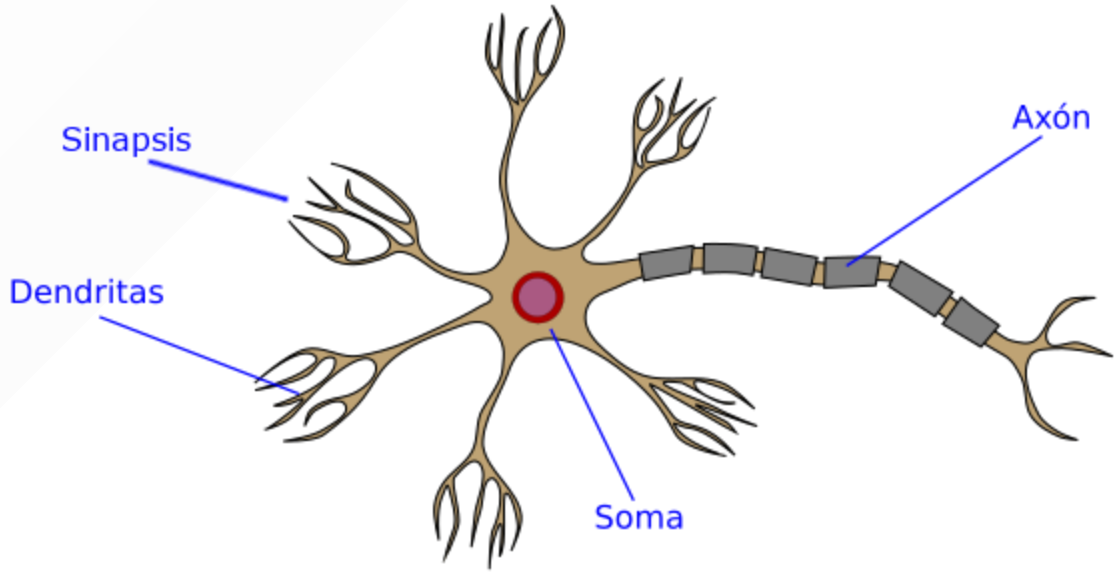
5.2.1.1 La neurona Biológica

La neurona se considera como **la unidad funcional básica del sistema nervioso**, que en los organismos vivos es el encargado de su mantenimiento vital. El sistema nervioso tiene como funciones principales el funcionamiento global del ser vivo y su relación con el entorno.

Las células nerviosas cuentan, como el resto de las células, con un cuerpo celular y un **núcleo** donde se encuentra la **información genética** de la célula o ADN. Pero además cuentan con una serie de prolongaciones con las que se intercomunican entre sí.

5.2.1.2 Partes de la neurona biológica

- **El axón** es la ramificación de salida de la neurona. A través de él se propagan los impulsos nerviosos.
- **Las dentritas** son ramificaciones de entrada para la célula
- Entre dentrita y axón se encuentra la **sinapsis** que es un líquido semiconductor que permite circular un cierto grado del impulso



Cuando el impulso llega a los extremos del axón, éste libera unas sustancias químicas denominadas **neurotransmisores** que se generan en los terminales del axón denominados **botones sinápticos**. Estos neurotransmisores intentan cruzan el espacio sináptico. Pero este le ofrece una resistencia dieléctrica que debe ser superada por los neurotransmisores para poder alcanzar las dendritas de la neurona receptora.

A pesar de lo que pueda parecer, la capacidad de cómputo principal de las neuronas no está en su núcleo sin en sus **interconexiones**. Es decir en la Sinapsis. La potencia de cerebro humano se obtiene por el número de interconexiones que se producen entre las neuronas. Una neurona se conecta con unas 10.000 neuronas y en el cerebro humano hay varios cientos de millones de ellas.

5.2.2 Redes de neuronas artificiales

Al igual que el cerebro humano, sustentan su capacidad de cálculo en las interconexiones. Pero debemos tener en cuenta que la neurona artificial es una analogía matemática y que su complejidad sináptica es mucho menor que la biológica.

Definimos Neurona Artificial como un elemento que posee un estado interno (S) denominado **nivel de activación** que simula la resistencia dieléctrica de la sinapsis. Además, recibe un conjunto de señales que le permiten cambiar de estado a través de una conexión con otras neuronas, que influyen en el nivel de activación con señales positivas y negativas (Simulando los **neurotransmisores**)

La función matemática que permite cambiar el estado de activación dependiendo de las entradas se denomina **función de activación**

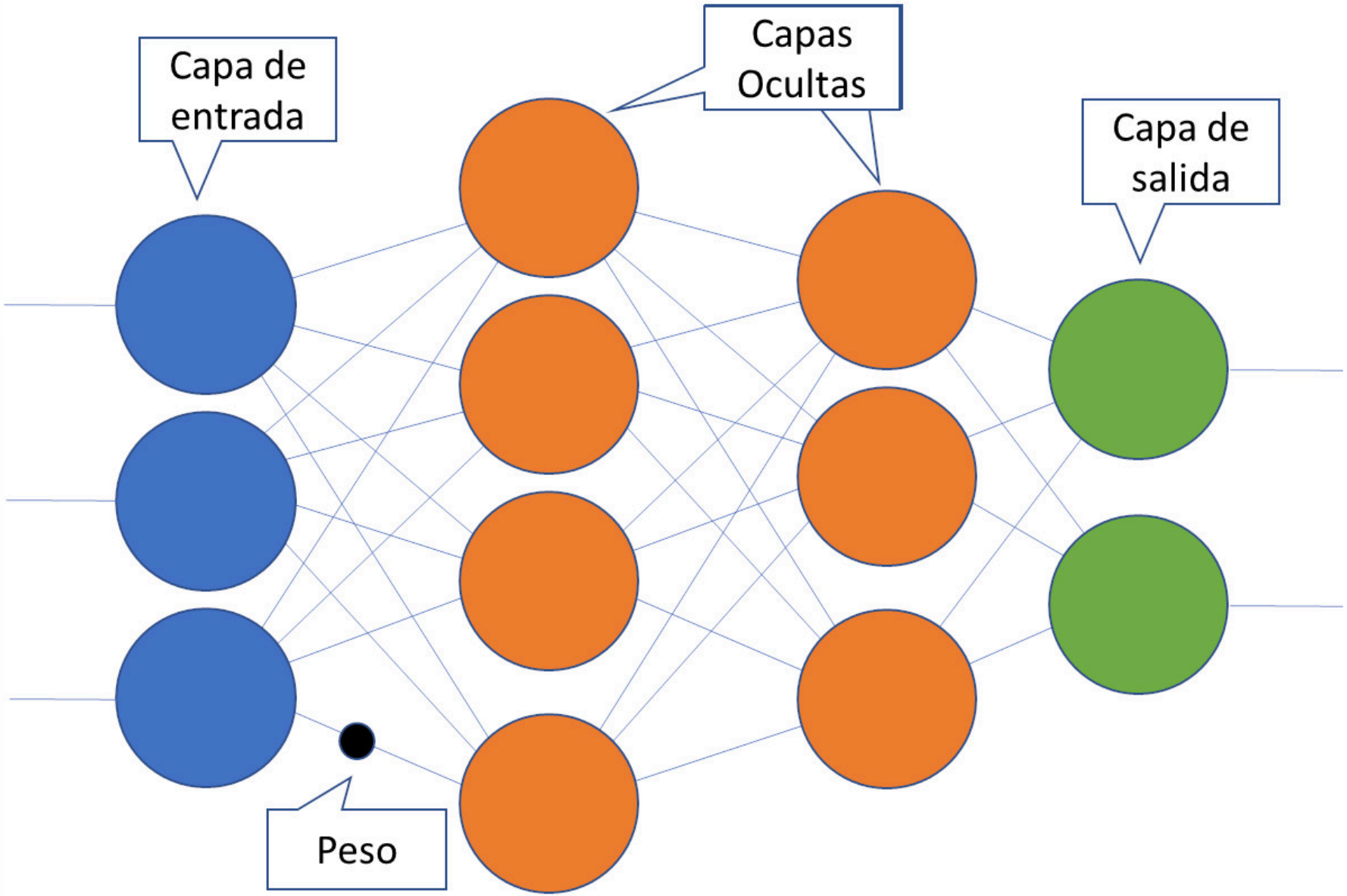
Cada entrada de la neurona es **multiplicada por un peso** que es modificable durante el aprendizaje. Cada peso permite que llegue una cantidad mayor o menor de la señal procedente de otra neurona.

Una red neuronal es una colección de neuronas cuya salida está conectada a las entradas de otras neuronas formando un **grafo dirigid** G(V,A) donde los vértices (V) son las neuronas y las aristas (A) son las conexiones entre ellas.

5.2.2.1 Tipos de neuronas

- **Neuronas de entrada**: Son las encargadas de recibir los datos del exterior de la red e inician la propagación de dichos datos por toda la estructura
- **Neuronas ocultas**: neuronas que permiten enlazar las neuronas de entrada y las neuronas de salida, aunque también pueden estar conectadas a otras neuronas ocultas
- **Neuronas de salida**: Son las que muestran al exterior los resultados obtenidos después de que la entrada haya recorrido toda la red

Capa: colección de neuronas que tienen una función común


Capa de entrada
Capas Ocultas
Capa de salida
Peso

Normalmente en cada neurona se hace la suma ponderada de los pesos para calculara su activación (aunque tambien se usa max y min)

Podemos definir por lo tanto de forma matemática una red de neuronas como una ecuación vectorial, de la siguiente forma:

$$\vec{S} = F(F(F(\vec{X} \cdot W_1) \cdot W_2) \dots W_i)$$

Si la **función de activación es lineal**, añadir más capas en la red es irrelevante y se podría crear una red de una sola capa oculta equivalente.

Pero si la función de activación no es lineal (lo habitual), entonces añadir más capas si tiene sentido.

5.2.2.2 Funciones de activación más usadas

- La función escalón donde α es el valor umbral de activación:

$$y(x) = \begin{cases} 1 & si \quad x \geq \alpha \\ \\ -1 \quad o \quad 0 & si \quad x \leq \alpha \end{cases}$$

- función sigmoidal (logística)

$$y(x) = \frac{1}{1 + e^{-x\rho}}$$

- La tangente hiperbólica:

$$y(x) = tanh(x) = \frac{senh(x)}{cosh(x)} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- La función rectificadora (ReLU):

$$y(x) = max(0, x)$$

Las funciones sigmoidal e hiperbólicas son diferenciables y monótonas y son no lineales, valores que nos interesan.

5.2.2.3 Aprendizaje de la red

Que la red aprenda consiste en obtener el valor de los pesos que conectan las diferentes capas que permitan aproximar lo más posible la función que queremos aprender.

Como la función no la conocemos, debemos inferirla a través de los ejemplos que se proporcionan a la red.

El principal mecanismo de aprendizaje de una red de neuronas es el **algoritmo de retropropagación** que veremos a continuación como parte del **perceptrón multicapa**, que nos servirá de modelo de red de neuronas base.

5.2.4 Perceptron multicapa

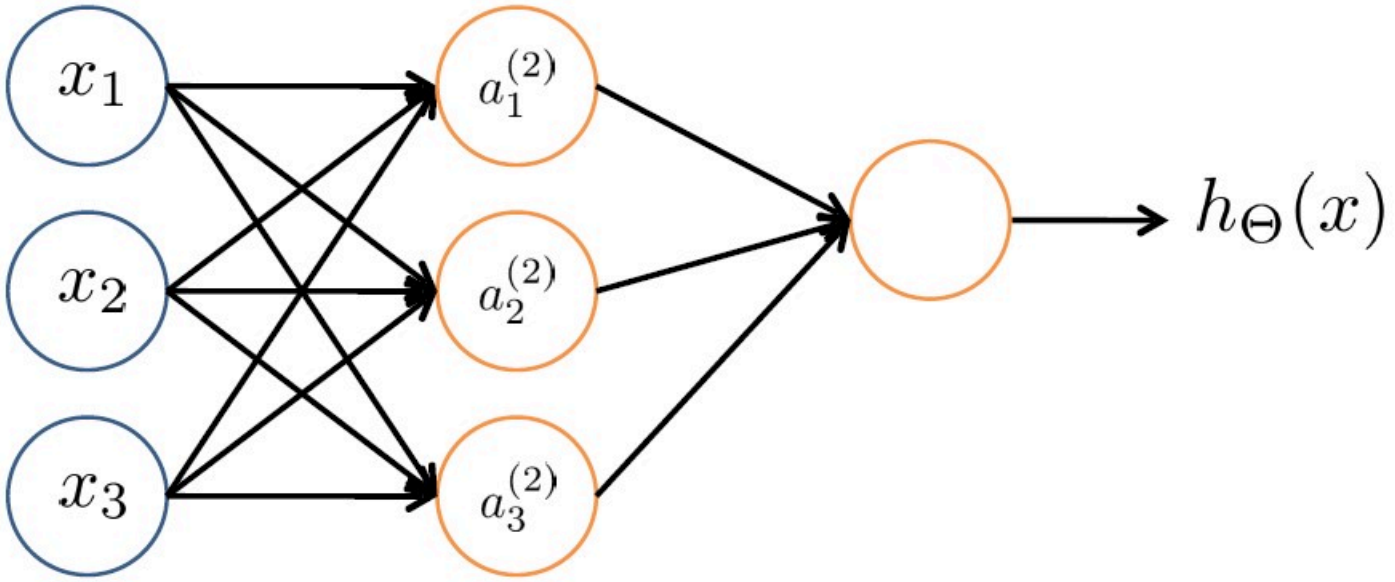
Es una generalización multicapa del **perceptrón** definido por Rosenblattd (1962). Usa como función de entrada la suma ponderada y como función de activación la que el usuario establezca (inicialmente el perceptron de Rosenblattd usaba la función escalón)

Rumelhartt, Hilton y Willians en 1989 presentaron un mecanismo que permitía dicha propagación del error por todos los pesos de las conexiones, permitiendo crear modelos de redes de más de una capa.

El **perceptrón multicapa** está demostrado que es un **aproximador universal** de cualquier función.

5.2.4.1 Pero antes de entrenar la red...

Veamos cuál es la primera fase de una red neuronal: la **propagación hacia delante (feedforward)** .



Sea a_i^j la activación de la unidad i en la capa j y Θ^j la matriz de pesos que controla la activación desde la capa j a la capa j+1.

La **capa de entrada** tendrá tantas neuronas como datos de entrada.

La **capa oculta** podría tener cualquier número de neuronas, pero para este ejemplo supondremos que tiene el mismo número que la entrada.

Para cada conexión desde la entrada a la capa oculta hay un peso w que conecta cada neurona de la capa de entrada con otra neurona de la capa oculta. La activación es la suma de la entrada multiplicada por los pesos y , añadiendo un parámetro más llamado **umbral/bias**. Es como si hubiera otra entrada siempre a 1

En el ejemplo anterior

$$a_1^2 = g(\Theta_{1,1}^1 x_1 + \Theta_{1,2}^1 x_2 + \Theta_{1,3}^1 x_3 + b_1^1)$$

$$a_2^2 = g(\Theta_{2,1}^1 x_1 + \Theta_{2,2}^1 x_2 + \Theta_{2,3}^1 x_3 + b_2^1)$$

$$a_3^2 = g(\Theta_{3,1}^1 x_1 + \Theta_{3,2}^1 x_2 + \Theta_{3,3}^1 x_3 + b_3^2)$$

El umbral o vias es como si establecemos ($x_0 = 1$)

$$a_1^2 = g(\Theta_{1,0}^1 + \Theta_{1,1}^1 x_1 + \Theta_{1,2}^1 x_2 + \Theta_{1,3}^1 x_3)$$

$$a_2^2 = g(\Theta_{2,0}^1 + \Theta_{2,1}^1 x_1 + \Theta_{2,2}^1 x_2 + \Theta_{2,3}^1 x_3)$$

$$a_3^2 = g(\Theta_{3,0}^1 + \Theta_{3,1}^1 x_1 + \Theta_{3,2}^1 x_2 + \Theta_{3,3}^1 x_3)$$

Para calcular a_1^3 (La capa de salida))...

$$a_1^3 = g(\Theta_{1,0}^2 + \Theta_{1,1}^2 a_1^2 + \Theta_{1,2}^2 a_2^2 + \Theta_{1,3}^2 a_3^2)$$

Nótese que a_0^2 no se calcula porque es como si la activación siempre estuviese a 1.

Feedforward iterativo:

```
#W weights, X input, A! activation of Hidden layer, G function
for j in range(len(A1))
  for i in range(len(X))
    A1[j] = W[j,i]*X[i]
  A1[j]=g(A1[j])
#Same for A2, A3.... An
```

Feedforward iterativo con biases no incluidos en la matriz de pesos

```
#W weights, X input, A! activation of Hidden layer, G function, b biases of layer 0 to layer 1
for j in range(len(A1))
  for i in range(len(X))
    A1[j] = W[j,i]*X[i] + b[i]
  A1[j]=g(A1[j])
#Same for A2, A3.... An
```

5.2.4.2 Forward propagation (feedforward)

Si generalizamos para cualquier número de capas y número de neuronas por capa:

$$a_i^j = g((\sum_{k=1}^{|a^{j-1}|} \Theta_{i,k}^j a_k^{j-1}) + b_i^j) \forall i \in [1..|a^j|], j \in [1..|C|]$$

Donde C son las capas del modelo, |C| es el número de capas, $|a_j|$ es el número de neuronas en la capa j, $|a^{j-1}|$ número de neuronas en la capa anterior y g la función de activación (ReLU, sigmoidal, etc).

5.2.4.3 ¿Cómo codifico la salida?

Todos los valores de una NN son números reales, ¿cómo creo un clasificador?

En teoría, cualquier codificación de la salida real sería válida, en la práctica, se utiliza como codificación que cada neurona de salida de la red representa una clase.

Otras condifcaciones: La codificación binaria, por ejemplo, produce peores resultados y no expresa la salida en forma de probabilidad, lo cual es bastante útil para razonar sobre la salida de la red a posteriori.

5.2.5 Normalización

La red neuronal funciona mejor con datos normalizados, ya sea en el rango 0..1 o en el rango -1..1. Por lo tanto, es un requisito previo que los valores de entrada de la red estén normalizados, utilizando z-Score o dividiendo por el valor máximo. Podemos utilizar Sklearn para normalizar los datos

```
from sklearn import preprocessing
import numpy as np
x_array = np.array([2,3,5,6,7,4,8,7,6])
normalized_arr = preprocessing.normalize([x_array])
print(normalized_arr) ## [[0.11785113 0.1767767  0.29462783 0.35355339 0.41247896 0.23570226
## 0.47140452 0.41247896 0.35355339]]
...
preprocessing.MinMaxScaler()
```

5.2.6 El algoritmo de retropropagación

La idea principal que sustenta al Perceptron es su mecanismo de aprendizaje basado en la regla delta (inicialmente introducida en la red ADALINE) pero generalizada a n capas ocultas.

La regla delta se basa en el **error cuadrático medio** que se produce en las salidas de la red si son comparadas con la salida real del dato de entrenamiento. El error de toda la red E es el siguiente, donde e(p) es el error producido por el patrón p:

$$E = \frac{1}{N} \sum_{p=1}^{N} e(P)$$

Como en los algoritmos que hemos visto antes, necesitamos calcular una función de coste J. Partimos de la función de coste de regresión logística:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} [y_i log(y'_i) + (1 - y_i) log(1 - y'_i)]$$

Pero podemos tener más de una salida y'_i = i-ésima salida

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} [y_{k,i} log(y'_{k,i}) + (1 - y_{k,i}) log(1 - y'_{k,i})]$$

La suma del error de todas las K neuronas de salida es el error producido por un ejemplo, el error medio - la media de los m

5.2.6.1 ¿Cómo modificar los pesos de la RNA para aprender?

El descenso de gradiente es tu amigo :).

El descenso de gradiente consiste en calcular la derivada de la función de error. Es un algoritmo de optimización (similar a Hill Climbing o Genéticos) y permite optimizar los pesos de una red.

$$\frac{\partial J(\Theta)}{\partial \Theta_{i,j}^{l}} = \sum_{k=1}^{m} \delta_{i,k}^{l+1} a_{j,k}^{l}$$

La derivada parcial de un peso $(\Theta_{j,i}^{l})$ que conecta la neurona j de la capa l+1 con la neurona i de la capa l de la matriz de pesos Θ^{l} = al

En la penúltima capa (L-1), sin embargo, debemos propagar el error a través de la red utilizando los pesos.

$$\delta_i^{L-1} = \Theta_{j,i}^{L-1} \delta_j^L g'(a_i^{L-1})$$

Es decir: el error de la capa siguiente X por el peso que conecta la neurona de la que calculamos su delta, por la derivada la neurona donde calculamos delta (g'), que en el caso de la función logistica es:

$$g'(a_i^l) = a_i^l(1 - a_i^l)$$

En otras palabras, la derivada de la función consiste en multiplicar la activación obtenida por la neurona, por 1 menos la activación obtenida por la neurona.

Generalizando para cualquier capa, el factor delta del error puede expresarse del siguiente modo:

$$\delta_i^l = \Theta_{j,i}^l \delta_j^{l+1} g'(a_i^l)$$

Por lo tanto, la disminución del gradiente para cualquier peso en cualquier capa puede expresarse como: $\Theta_{j,i}^l = \Theta_{j,i}^l - \alpha \frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l}$ donde:

$$\frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l} = \frac{1}{m} \sum_{k=1}^{m} \delta_j^{l+1}(k) a_i^l(k)$$

NOTA: algunas implementaciones no dividen entre m y otras no calculan la derivada de la función en la última capa.

Y los deltas del resto de capas se calcularían del siguiente modo:
Regla Delta Generalizada Para todas las capas desde la 2 hasta la L-1 (las capas ocultas) para el ejemplo k-ésimo (k) de entrenamiento:

$$\delta_i^l(k) = \sum_{j=1}^{size(a^{l+1})} (\Theta_{j,i}^l \cdot \delta_j^{l+1}(k) \cdot g'(a_i^l(k)))$$

Donde la derivada será la siguiente:

$$g'(a_i^l(k)) = a_i^l(k) * (1 - a_i^l(k))$$

Para todos los k ejemplos de entrenamiento, y para cada una de las i neuronas de la capa l correspondiente. (Un delta por cada neurona)

Resumen de ecuaciones del backpropagation:

Reglas delta:

Para la ultima capa:

$$\delta_j^L(k) = y_j\prime(k) - y_j(k)$$

Para el resto de capas ocultas:

$$\delta_i^l(k) = \sum_{j=1}^{size(a^{l+1})} (\Theta_{j,i}^l \delta_j^{l+1}(k) g'(a_i^l(k)))$$

5.2.6.2 Algoritmo:

- Inicializar los pesos y umbrales de la red con valores aleatorios.
- Para cada patrón de entrada
  - Cargar las entradas en las neuronas de entrada.
  - Propagar la entrada a través de la red (feedforward).
  - Calcular el error de ese patrón
  - Calcular todos los δ para todas las conexiones de la red.
  - Calcular los pesos partiendo de la salida hacia la entrada (Backpropagation)
  - Almacenar el error cometido
- Calcular el error medio

5.2.6.3 Umbrales o biases

Los umbrales o bias pueden ir acuplados a la matriz de pesesos estableciendolos como una entrada del a capa anterior (una red de nueronas más) con valor de activación de siempre 1. De esta forma, se actualizarán automáticamente tambien con las fórmulas anteriores. En caso de que se saquen de la matriz de pesos, la regla delta generalizada para los humbrales b se puede resumir así:

$$b^{c+1} = b^{c+1} + \alpha \cdot \delta^{c+1}$$

Tambien puede calcularse vectorialmentecreando una nuerona artificial en cada capa (menos la ultima) que siempre vale 1

5.2.7 Regularization

Si aplicamos la regularización L2, el error es ligeramente distinto. Hay que sumarle la suma de todos los pesos de la red al cuadrado

$$JL2(\Theta) = J(\Theta) + \lambda \frac{1}{2m} \sum_{l=1}^{L-1} \sum_{j=1}^{Dim(l)} \sum_{i=1}^{Dim(l+1)} (\Theta_{i,j}^{l})^2$$

La regularización L1 es similar, pero calculando el valor absoluto. El término gradiente en la regularización (derivada del anterior):

$$\partial JL2(\Theta) = \partial J(\Theta) + \lambda \frac{1}{m} \sum_{l=1}^{L-1} \sum_{j=1}^{Dim(l)} \sum_{i=1}^{Dim(l+1)} (\Theta_{i,j}^{l})$$

5.2.8 Hiperparámetros de un perceptrón multicapa

- Las capas de entrada y salida las determina el problema y la codificación elegida del mismo
  - Capa de entrada: típicamente 1 por cada atributo de los ejemplos. Las entradas mejor si están normalizadas a valores (0..1)
  - Capa de salida: en función de cómo codifiquemos la salida:
    - Regresión: una única salida o tantas como valores queramos predecir.
    - Clasificación: codificar la salida, una salida por cada clase (la

5.2.8.1 Hiperparámetros de un perceptrón

- Las capas ocultas si son cuestiones que decide el diseñador de la red, cuantas y que neuronas tiene cada una
- La función de activación
- El factor de aprendizaje α, normalmente suele funcionar valores pequeños, pero depende del problema y puede ser dinámico (al principio mas grande y después más pequeño o viceversa)
- Número de iteraciones máximas: número de veces que se presentan los datos de entrenamiento a la red
- Criterio de parada (si el error de validación llega a un valor con el que estamos cómodos)

5.2.8.2 Caracteristicas importantes de las RN

Como todos los algoritmos de ML, las redes de neuronas pueden **sobreadaptarse a los datos de entrenamiento**. Hay que vigilar el entrenamiento y la validación. Si el error de validación comienza a crecer mucho es recomendable parar. También puede converger prematuramente en mínimos locales y estancarse.

Son bastante **buenas tolerando el ruido en la entrada** pero por contra son **algoritmos de caja negra**. Es complicado entender el modelo subyacente lo que dificulta su utilización en entornos donde se necesita validar el modelo (medicina por ejemplo)

Necesitan **muchos datos para que el entrenamiento funcione**

5.2.8.3 ¿Cómo encontramos los parámetros correctos?

Podemos buscar los parámetros que mejor se adaptan al problema. Existen diferentes técnicas como:

- Grid search:
- Random search
- Usando algoritmos genéticos.
- Algoritmos de descenso de gradiente

Sólo podamos explorar una pequeña parte del espacio de los hiperparámetros en un tiempo razonable.

5.2.8.4 ¿Cuando parar una red?

- **Epoch** (época) se refiere a utilizar una vez todo el conjunto de entrenamiento. Suele haber un número máximo.

Se puede terminar el proceso automáticamente cuando no se producen mejoras del entrenamiento durante un cierto número de iteraciones.

5.2.8.5 Ejemplo en SKLearn

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
X, y = make_classification(n_samples=100, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,random_state=1)
clf = MLPClassifier(random_state=1, max_iter=300, verbose=True).fit(X_train, y_train)
clf.predict_proba(X_test[:1])
y_pred = clf.predict(X_test[:, :])
print(y_test)
print(y_pred)
clf.score(X_test, y_test)
```

```
Iteration 1, loss = 0.71063329
Iteration 2, loss = 0.69638254
Iteration 3, loss = 0.68245678
Iteration 4, loss = 0.66885112
Iteration 5, loss = 0.65553494
Iteration 6, loss = 0.64257111
Iteration 7, loss = 0.62991247
Iteration 8, loss = 0.61751799
Iteration 9, loss = 0.60539429
```

Bibliografia

- Viñuela, P. I., & León, I. M. G. (2004). Redes de neuronas artificiales: un enfoque práctico. Prentice Hall.
- Bagén, L., Toni, B. R., & Anna, C. R. (2019). Deep learning: principios y fundamentos. Deep learning, 1-260.
- Buduma, N., Buduma, N., & Papa, J. (2022). Fundamentals of deep learning. " O'Reilly Media, Inc.".